

Week 07: Building & Training a Modern LLM From Scratch

RMSNorm · RoPE · GQA · SwiGLU — The Full Decoder Block in PyTorch

WEEK

07

DATE

27 FEB 2026

MODEL

MINILLM

 PYTORCH

MODERN DECODER BLOCK

Every component used in production LLMs like LLaMA, Mistral, and Gemma — implemented line by line.

Quick Links

Resource	Link
 Lecture slides	Google Drive
 Colab notebook	Google Colab

At a Glance

What was built

A **decoder-only character-level language model** (MiniLLM) with all four modern upgrades:

Component	What It Does	Why It Matters
RMSNorm (pre-norm)	Normalizes activations via root-mean-square	Simpler & faster than LayerNorm; used in LLaMA/Mistral
RoPE	Rotates Q/K vectors by position	Encodes relative distance, not absolute index
GQA	Shares K/V heads across query groups	4× KV-cache savings at inference time
SwiGLU FFN	Gated feed-forward with smooth activation	Better gradients + learned feature gating
Residual + Weight Tying	Shortcuts + shared embedding/head weights	Stable deep training + fewer parameters

Baseline model config (from notebook run)

Hyperparameter	Value	Notes
vocab_size	65	Unique chars in Tiny Shakespeare

Hyperparameter	Value	Notes
d_model	256	Width of residual stream
n_layers	4	Depth of transformer stack
n_heads	8	Query attention heads
n_kv_heads	2	KV heads (GQA ratio 4:1)
head_dim	32	= d_model / n_heads
ffn_hidden_dim	680	SwiGLU expansion width
max_seq_len	256	Context window
Total params	2,763,264	~11.1 MB in fp32

Baseline training run (Tesla T4)

Metric	Value
Steps	3,000
Batch size	64
Context length	256
Learning rate	3e-4
Runtime	673s (~11.2 min)
Final train loss	1.0163
Final val loss	1.5086

Week 06 → Week 07 Bridge

Week 06 was **theory**: "What are the modern upgrades to the vanilla transformer?" Week 07 is **practice**: "Build every single one of them in PyTorch."

Component	Week 06 (Concept)	Week 07 (Implementation)
Normalization	RMSNorm + pre-norm stability	Custom <code>RMSNorm</code> module
Position	RoPE for relative position	<code>precompute_rope_freqs</code> + <code>apply_rope</code>
Attention	MHA → GQA for KV savings	<code>GroupedQueryAttention</code> + <code>repeat_kv</code>
FFN	ReLU → SwiGLU gating	Custom <code>SwiGLU</code> module
Full block	Modern decoder block diagram	<code>TransformerBlock</code> + <code>MiniLLM</code> end-to-end

Analogy: Week 06 was reading the blueprint for a modern engine. Week 07 is machining every part and assembling it on the bench.

 Table of Contents

1. Setup and Data

2. RMSNorm and Dropout

3. RoPE (Rotary Positional Embeddings)

4. GQA (Grouped Query Attention)

5. SwiGLU Feed-Forward Network

6. Model Assembly

7. Training and Evaluation

8. Generation (Autoregressive Sampling)

9. Debugging and Sanity Checks

10. Extension Roadmap

11. Quick Revision Card

1. Setup and Data

1.1 Environment

```
device = "cuda" if torch.cuda.is_available() else "cpu"
```

Observed in notebook:

- Device: cuda

• GPU: Tesla T4 (16 GB VRAM, ~65 TFLOPS FP16)

1.2 Dataset: Tiny Shakespeare

Property	Value
Source	karpathy/char-rnn
Total characters	1,115,394
Content	All 36 Shakespeare plays concatenated
Format	Raw text with dialog structure (CHARACTER:\nLine...)

Why Tiny Shakespeare?

It's small enough to train in minutes on a free GPU, yet structured enough (dialog, verse, prose) to produce visually recognizable output quality differences.

1.3 Character-level tokenizer

```
chars = sorted(set(text))
vocab_size = len(chars) # 65

char_to_idx = {c: i for i, c in enumerate(chars)}
idx_to_char = {i: c for i, c in enumerate(chars)}
```

```
def encode(s): return [char_to_idx[c] for c in s]
def decode(ids): return "".join([idx_to_char[i] for i in ids])
```

Observed:

```
Vocabulary size: 65
Characters: \n !$&',-.3:;?ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz
"Hello" -> [20, 43, 50, 50, 53]
```

Why character-level here (and not BPE)?

	Character-Level	BPE / SentencePiece
Complexity	Trivial — just <code>sorted(set(text))</code>	Requires training merge rules
Vocab size	Tiny (65)	Thousands to hundreds of thousands
Sequence length	Longer (1 char = 1 token)	Shorter (1 word ≈ 1–3 tokens)
Focus	Architecture internals	Production language quality
Debugging	Easy — every token is visible	Harder — subword boundaries are opaque

Analogy: Character-level tokenization is like learning to drive in a parking lot — you focus on the steering mechanics, not the highway traffic. BPE is the highway.

1.4 Train / validation split

```
data = torch.tensor(encode(text), dtype=torch.long)
n = int(0.9 * len(data))
train_data = data[:n]
val_data = data[n:]
```

Split	Tokens	Percentage
Train	1,003,854	90%
Validation	111,540	10%

Note: This is a simple sequential split (first 90% → train, last 10% → val), not a random shuffle. That's fine for language modeling — we don't need i.i.d. samples; the model should generalize across Shakespeare's entire corpus.

1.5 Batching and autoregressive targets

```
def get_batch(split, batch_size, context_length):
    d = train_data if split == "train" else val_data
    ix = torch.randint(len(d) - context_length, (batch_size,))
```

```
x = torch.stack([d[i:i+context_length] for i in ix])
y = torch.stack([d[i+1:i+context_length+1] for i in ix])
return x.to(device), y.to(device)
```

Key idea:

- y is x shifted right by **exactly one token**
- At every position, the model learns: "given all tokens so far, predict the next one"

Concrete example (from the text **To be,**):

Position	What the model sees (context)	What it must predict (target)
0	T	o
1	To	
2	To	b
3	To b	e
4	To be	,

Why this works: A single sequence of length L gives us L independent next-token prediction problems for free. With batch size 64 and context length 256, each step gives us $64 \times 256 = 16,384$ token predictions to learn from.

2. RMSNorm and Dropout

2.1 Why RMSNorm over LayerNorm?

LayerNorm (the original transformer normalizer) does three things:

1. Subtract the mean (center the distribution)
2. Divide by the standard deviation (scale to unit variance)
3. Apply learned scale (γ) and shift (β) parameters

RMSNorm simplifies this dramatically — it drops step 1 (mean subtraction) and step 3's shift (β):

2. RMSNorm and Dropout

2.1 Why RMSNorm over LayerNorm?

LayerNorm (the original transformer normalizer) does three things:

- 1. Subtract the mean (center the distribution)
- 2. Divide by the standard deviation (scale to unit variance)
- 3. Apply learned scale (γ) and shift (β) parameters

RMSNorm simplifies this dramatically — it drops step 1 (mean subtraction) and step 3's shift (β):

$$\text{RMS}(x) = \sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2 + \epsilon}$$
$$y_i = \frac{x_i}{\text{RMS}(x)} \cdot \gamma_i$$

Intuition: LayerNorm asks "how far is each value from the average?" RMSNorm asks the simpler question "how big are the values overall?" The research finding is that the mean-centering step wasn't contributing much — removing it makes the math simpler with no quality loss.

Side-by-side comparison:

Property	LayerNorm	RMSNorm
Mean subtraction	✔ Yes	✗ No
Bias/shift term (β)	✔ Yes	✗ No
Learned parameters	$\gamma + \beta$ (2d)	γ only (d)
Compute cost	Higher	Lower
Used in modern LLMs	Less common	✔ LLaMA, Mistral, Gemma

Analogy: Imagine normalizing exam scores. LayerNorm first shifts everyone's score so the class average is zero, then scales so the spread is 1. RMSNorm skips the shifting step — it just scales everyone's score by the overall magnitude. Turns out, for transformer internals, that's enough.

Intuition: LayerNorm asks "how far is each value from the average?" RMSNorm asks the simpler question "how big are the values overall?" The research finding is that the mean-centering step wasn't contributing much — removing it makes the math simpler with no quality loss.

Side-by-side comparison:

Property	LayerNorm	RMSNorm
Mean subtraction	✔ Yes	✗ No
Bias/shift term (β)	✔ Yes	✗ No
Learned parameters	$\gamma + \beta$ (2d)	γ only (d)
Compute cost	Higher	Lower
Used in modern LLMs	Less common	✔ LLaMA, Mistral, Gemma

Analogy: Imagine normalizing exam scores. LayerNorm first shifts everyone's score so the class average is zero, then scales so the spread is 1. RMSNorm skips the shifting step — it just scales everyone's score by the overall magnitude. Turns out, for transformer internals, that's enough.

2.2 Implementation

```
class RMSNorm(nn.Module):
    def __init__(self, dim, eps=1e-6):
        super().__init__()
        self.eps = eps
        self.weight = nn.Parameter(torch.ones(dim)) #  $\gamma$ , initialized to 1

    def forward(self, x):
        rms = torch.sqrt(x.pow(2).mean(dim=-1, keepdim=True) + self.eps)
        return (x / rms) * self.weight
```

Line-by-line breakdown:

Line	What it does
<code>x.pow(2)</code>	Square every element
<code>.mean(dim=-1, keepdim=True)</code>	Average across the feature dimension → mean of squares
<code>+ self.eps</code>	Add tiny constant (1e-6) to prevent division by zero
<code>torch.sqrt(...)</code>	Take the root → root-mean-square
<code>x / rms</code>	Normalize: divide each element by the RMS
<code>* self.weight</code>	Scale by learned per-dimension weight γ

Observed demo output:

```
RMSNorm demo:
Input  - mean: -0.015, std: 1.004
Output - mean: -0.000, std: 1.008
Parameters: just a scale vector of size torch.Size([8])
```

2.3 Pre-norm vs. Post-norm placement

This model uses **pre-norm** (normalize *before* each sublayer), not post-norm (normalize *after*):

```
Pre-norm:  x → Norm → Sublayer → + x      ← used here (and in LLaMA, GPT-3, etc.)
Post-norm: x → Sublayer → + x → Norm      ← original transformer (Vaswani 2017)
```

Why pre-norm wins: Pre-norm puts the residual connection *outside* the normalization. This means gradients can flow through the residual path completely unimpeded — they don't have to pass through the normalization layer during backprop. Result: much more stable training for deep networks, enabling training of 100+ layer models without careful LR tuning.

2.4 Dropout for regularization

```
DROPOUT = 0.2
```

Mode	Behavior
Training	Randomly zero ~20% of activations (and scale remaining by 1/0.8)
Inference (<code>model.eval()</code>)	No dropout — all neurons active

Purpose:

- Forces the network to not rely on any single feature (*co-adaptation*)
- Acts as implicit ensemble (each forward pass uses a different sub-network)
- Critical for small datasets like Tiny Shakespeare where overfitting is easy

Analogy: Dropout is like studying for an exam by randomly covering parts of your notes each study session. You're forced to learn the material from multiple angles instead of memorizing one specific path.

3. RoPE (Rotary Positional Embeddings)

3.1 The Problem: Transformers Have No Sense of Order

Self-attention computes $Q \cdot K^T$ — a dot product that is **permutation-invariant**. Shuffle the input tokens, and the attention scores don't change. Without positional information, "the cat sat" and "sat cat the" are identical to the model.

Three generations of positional encoding:

Generation	Method	Limitation
1st (Original Transformer)	Add fixed sinusoidal vectors	Absolute position only; <code>pos=5</code> always gets the same encoding
2nd (GPT-2)	Learned position embeddings	Still absolute; limited to trained <code>max_seq_len</code>
3rd (Modern: RoPE)	Rotate Q and K by position	Encodes relative distance ; can extrapolate beyond training length

3.2 The Key Insight: Rotation Encodes Relative Distance

The Clock Analogy: Think of each dimension pair in Q/K as a hand on a clock. At position `m`, the hand rotates by angle $m \cdot \theta$. When we compute $Q \cdot K^T$, the rotation cancels out the *absolute* positions and leaves only the *difference* — the angle *between* the two positions. The gap between 3 o'clock and 5 o'clock is the same as the gap between 7 o'clock and 9 o'clock. **Only the relative distance matters.**

3.3 Core Formulas

For each dimension pair (`2i`, `2i+1`):

3.3 Core Formulas

For each dimension pair $(2i, 2i+1)$:

Step 1: Compute the frequency for this pair: $\theta_i = \text{base}^{-\frac{2i}{d}}$ (base = 10000 by default)

Step 2: Rotate the 2D vector at position m :

$$x'_{2i} = x_{2i}\cos(m\theta_i) - x_{2i+1}\sin(m\theta_i)$$

$$x'_{2i+1} = x_{2i}\sin(m\theta_i) + x_{2i+1}\cos(m\theta_i)$$

This is literally a **2D rotation matrix** applied to each pair of dimensions.

Frequency hierarchy — the multi-resolution trick:

Dimension Pair	Frequency (θ)	Rotation Speed	Sensitivity
Low dims (pair 0, 1)	High	Fast oscillation	Short-range patterns (adjacent chars)
Mid dims	Medium	Moderate	Medium-range patterns
High dims (pair $d/2-1$)	Low	Slow oscillation	Long-range patterns (distant context)

Analogy: RoPE is like a set of clock hands with different speeds — the second hand captures fine timing, the minute hand captures medium patterns, and the hour hand captures broad structure. Together, they encode position at every resolution simultaneously.

This is literally a **2D rotation matrix** applied to each pair of dimensions.

Frequency hierarchy — the multi-resolution trick:

Dimension Pair	Frequency (θ)	Rotation Speed	Sensitivity
Low dims (pair 0, 1)	High	Fast oscillation	Short-range patterns (adjacent chars)
Mid dims	Medium	Moderate	Medium-range patterns
High dims (pair $d/2-1$)	Low	Slow oscillation	Long-range patterns (distant context)

Analogy: RoPE is like a set of clock hands with different speeds — the second hand captures fine timing, the minute hand captures medium patterns, and the hour hand captures broad structure. Together, they encode position at every resolution simultaneously.

3.4 Precompute Cosine/Sine Tables

```
def precompute_rope_freqs(head_dim, max_seq_len, base=10000.0):
    freqs = 1.0 / (base ** (torch.arange(0, head_dim, 2).float() / head_dim))
    positions = torch.arange(max_seq_len).float()
    angles = torch.outer(positions, freqs) # [max_seq_len, head_dim // 2]
    return torch.cos(angles), torch.sin(angles)
```

Why precompute:

- The cos/sin values depend only on **position** × **frequency** — they never change during training
- Computing trig functions is expensive; doing it once and caching saves significant GPU time
- Tables are shape `[max_seq_len, head_dim//2] = [256, 16]` — tiny memory cost

3.5 Apply Rotation to Q and K

```
def apply_rope(x, cos, sin):
    seq_len = x.shape[2]
    cos = cos[:seq_len].unsqueeze(0).unsqueeze(0) # [1, 1, seq, hd//2]
    sin = sin[:seq_len].unsqueeze(0).unsqueeze(0)

    x1 = x[..., ::2] # even dims (x_{2i})
    x2 = x[..., 1::2] # odd dims (x_{2i+1})

    out1 = x1 * cos - x2 * sin # rotated even
    out2 = x1 * sin + x2 * cos # rotated odd
    return torch.stack([out1, out2], dim=-1).flatten(-2)
```

Critical detail:

- Apply to **Q** and **K** only — never to **V**
- V carries the *content* that gets mixed; it shouldn't be position-dependent
- The **stack + flatten** at the end interleaves the even/odd results back together

3.6 Why RoPE Beats Absolute Position Embeddings

Feature	Absolute Sinusoidal / Learned	RoPE
What's encoded	Fixed position index	Relative distance between tokens
How it's injected	Added to input embeddings	Multiplied into Q and K
Extrapolation	Fails beyond <code>max_seq_len</code>	Can generalize (rotation extends naturally)
Parameters	Learned table or fixed	Zero extra parameters
Where in modern LLMs	GPT-2, BERT	LLaMA, Mistral, Gemma, Qwen

4. GQA (Grouped Query Attention)

4.1 The KV Cache Bottleneck

In autoregressive generation, we decode one token at a time. At each step, we need the K and V vectors for **all previous tokens** — this is the **KV cache**.

Standard **Multi-Head Attention (MHA)**: each query head has its own independent K and V → KV cache grows linearly with `n_heads`.

GQA insight: Most of the "work" in attention is done by the query. K and V just provide the lookup table. Multiple query heads can **share** the same K/V pair without much quality loss — and the memory savings are enormous.

```
MHA (8Q, 8K, 8V): Full redundancy in KV
GQA (8Q, 2K, 2V): 4 query heads share each KV pair ← this model
```

MQA (8Q, 1K, 1V): All queries share one KV pair (extreme compression)

Analogy: MHA is like 8 students each having their own personal copy of a textbook. GQA is like giving 4 students one shared copy — they all look things up in the same book, but each writes their own essay (query). You save 4× on books with minimal loss in essay quality.

4.2 This Model's Head Configuration

Parameter	Value	Meaning
d_model	256	Total model width
n_heads	8	Number of query heads
n_kv_heads	2	Number of KV heads
head_dim	32	= 256 / 8
n_rep	4	= 8 / 2 (replication factor)

Each KV head is shared by **4 query heads**.

4.3 Projection-Level Savings

```
self.q_proj = nn.Linear(d_model, n_heads * head_dim, bias=False) # 256 → 256
self.k_proj = nn.Linear(d_model, n_kv_heads * head_dim, bias=False) # 256 → 64
self.v_proj = nn.Linear(d_model, n_kv_heads * head_dim, bias=False) # 256 → 64
```

Observed in notebook:

```
Q projection: [256, 256] ← full-rank
K projection: [64, 256] ← 4× smaller than Q
V projection: [64, 256] ← 4× smaller than Q
```

Parameter savings:

	MHA (8 KV heads)	GQA (2 KV heads)	Savings
K proj params	256 × 256 = 65,536	256 × 64 = 16,384	75%
V proj params	256 × 256 = 65,536	256 × 64 = 16,384	75%
KV cache size (per layer)	2 × 8 × seq × 32	2 × 2 × seq × 32	75%

4.4 The repeat_kv Helper

After computing the 2 KV heads, we need to "stretch" them to match the 8 query heads for the matmul:

```
def repeat_kv(x, n_rep):
    if n_rep == 1:
        return x # MHA case — no repetition needed
    b, n_kv, seq, hd = x.shape
    return (x[:, :, None, :, :] # insert repetition dim
            .expand(b, n_kv, n_rep, seq, hd) # broadcast (no memory copy!)
            .reshape(b, n_kv * n_rep, seq, hd)) # merge into head dim
```

Shape trace:

```
Before: [batch, 2, seq, 32]    ← 2 KV heads
Insert: [batch, 2, 1, seq, 32]
Expand: [batch, 2, 4, seq, 32] ← repeat each KV head 4 times
Merge:  [batch, 8, seq, 32]    ← now matches 8 query heads
```

Key implementation detail: `expand` uses **view semantics** — it doesn't actually copy memory. The 4 repetitions of each KV head are just different views of the same data. This makes GQA nearly free at runtime.

4.5 Full Attention Forward Pass

```
# Project to Q, K, V
q = self.q_proj(x).view(b, seq, 8, 32).transpose(1, 2) # [b, 8, seq, 32]
k = self.k_proj(x).view(b, seq, 2, 32).transpose(1, 2) # [b, 2, seq, 32]
v = self.v_proj(x).view(b, seq, 2, 32).transpose(1, 2) # [b, 2, seq, 32]

# Apply RoPE to Q and K (not V!)
q = apply_rope(q, rope_cos, rope_sin)
k = apply_rope(k, rope_cos, rope_sin)

# Expand KV heads to match Q heads
k = repeat_kv(k, 4) # [b, 8, seq, 32]
v = repeat_kv(v, 4) # [b, 8, seq, 32]

# Standard scaled dot-product attention
scores = (q @ k.transpose(-2, -1)) * (1.0 / math.sqrt(32))
scores = scores.masked_fill(causal_mask, float("-inf"))
weights = F.softmax(scores, dim=-1)
weights = F.dropout(weights, p=DROPOUT, training=self.training)
out = weights @ v

# Merge heads and project
out = out.transpose(1, 2).contiguous().view(b, seq, 256)
return self.o_proj(out)
```

4.6 The Causal Mask — Preventing Time Travel

Causal mask (4×4 example):

	pos 0	pos 1	pos 2	pos 3	
[True	False	False	False	]	← pos 0 sees only itself
[True	True	False	False	]	← pos 1 sees 0, 1
[True	True	True	False	]	← pos 2 sees 0, 1, 2
[True	True	True	True	]	← pos 3 sees all

- Positions marked **False** get **score** = $-\infty$, which becomes 0 after softmax
- **Without this mask:** the model would "cheat" by looking at future tokens during training
- **With this mask:** each position can only attend to past tokens and itself — exactly like generation time

Analogy: The causal mask is like writing an exam where you can only look at questions you've already answered. You can't peek ahead to question 5 while working on question 3.

5. SwiGLU Feed-Forward Network

5.1 From ReLU to Gated Activations

Classic FFN (original transformer):

$x \rightarrow \text{Linear_up (expand)} \rightarrow \text{ReLU} \rightarrow \text{Linear_down (contract)} \rightarrow \text{output}$

Problem: ReLU has a **hard cutoff** at zero. Everything below zero is killed to exactly 0, and the gradient is 0 for those neurons — they're effectively dead.

SwiGLU FFN (modern replacement):

$x \rightarrow \begin{matrix} [W_{\text{gate}} \cdot x \rightarrow \text{SiLU}] \\ \text{(gate path)} \end{matrix} \odot \begin{matrix} [W_{\text{up}} \cdot x] \rightarrow W_{\text{down}} \rightarrow \text{output} \\ \text{(value path)} \quad \quad \quad \text{(project)} \end{matrix}$

The key innovation is **gating**: one path decides *what to let through* (gate), another path provides *the actual content* (value), and they're combined with elementwise multiplication.

5.2 The Math

5. SwiGLU Feed-Forward Network

5.1 From ReLU to Gated Activations

Classic FFN (original transformer):

```
x → Linear_up (expand) → ReLU → Linear_down (contract) → output
```

Problem: ReLU has a **hard cutoff** at zero. Everything below zero is killed to exactly 0, and the gradient is 0 for those neurons — they're effectively dead.

SwiGLU FFN (modern replacement):

```
x → [W_gate · x → SiLU] ⊙ [W_up · x] → W_down → output
   (gate path)           (value path)   (project)
```

The key innovation is **gating**: one path decides *what to let through* (gate), another path provides *the actual content* (value), and they're combined with elementwise multiplication.

5.2 The Math

$$\text{SwiGLU}(x) = W_{\text{down}} \left(\text{SiLU}(W_{\text{gate}} \cdot x) \odot (W_{\text{up}} \cdot x) \right)$$

Where SiLU (Sigmoid Linear Unit, a.k.a. Swish) is:

$$\text{SiLU}(x) = x \cdot \sigma(x)$$

Why SiLU?

- Unlike ReLU, SiLU is **smooth everywhere** — no hard cutoff, no dead neurons
- Slightly allows small negative values through (non-monotonic near zero)
- Better gradient flow during backpropagation

Activation	Formula	Dead Neurons?	Gradient at x=0
ReLU	$\max(0, x)$	Yes — permanently	Undefined (0 or 1)
GELU	$x \cdot \Phi(x)$	No	~0.5
SiLU/Swish	$x \cdot \sigma(x)$	No	0.5

Where SiLU (Sigmoid Linear Unit, a.k.a. Swish) is shown above.

Why SiLU?

- Unlike ReLU, SiLU is **smooth everywhere** — no hard cutoff, no dead neurons
- Slightly allows small negative values through (non-monotonic near zero)
- Better gradient flow during backpropagation

Activation	Formula	Dead Neurons?	Gradient at x=0
ReLU	$\max(0, x)$	Yes — permanently	Undefined (0 or 1)
GELU	$x \cdot \Phi(x)$	No	~0.5
SiLU/Swish	$x \cdot \sigma(x)$	No	0.5

5.3 Implementation

```
class SwiGLU(nn.Module):
    def __init__(self, d_model, hidden_dim):
        super().__init__()
        self.w_gate = nn.Linear(d_model, hidden_dim, bias=False)
        self.w_up = nn.Linear(d_model, hidden_dim, bias=False)
        self.w_down = nn.Linear(hidden_dim, d_model, bias=False)

    def forward(self, x):
        gate = F.silu(self.w_gate(x))    # gate: what to let through
        up = self.w_up(x)                # up: the actual content
        return F.dropout(self.w_down(gate * up), p=DROPOUT,
                        training=self.training)
```

Dimensions for this model:

```
Input: 256 → [w_gate] → 680 → SiLU → ⊙
                                     ↓   → [w_down] → 256 → Output
Input: 256 → [w_up]   → 680 → ⊙
```

Why 3 matrices instead of the classic 2?

- Classic FFN: W_{up} (expand) + W_{down} (contract) = **2 matrices**
- SwiGLU: W_{gate} + W_{up} + W_{down} = **3 matrices**
- The extra matrix is the cost of gating. To keep total parameter count similar, modern LLMs often set $hidden_dim \approx (2/3) \times (4 \times d_model)$ instead of the classic $4 \times d_model$.
- Here: $hidden_dim = 680 \approx (2/3) \times 1024$

Analogy: The classic FFN is like a pipe — everything flows through the same path. SwiGLU is like a valve system: one pipe carries the content, another pipe controls the valve that decides how much of each feature gets through. This learned "feature gating" lets the network selectively amplify or suppress different features dimension by dimension.

6. Model Assembly

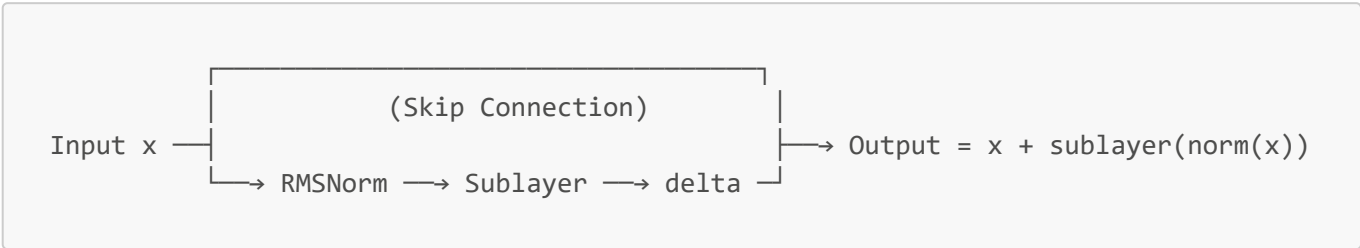
6.1 Transformer Block (Pre-Norm + Residual)

```
class TransformerBlock(nn.Module):
    def __init__(self, d_model, n_heads, n_kv_heads, ffn_hidden_dim):
        super().__init__()
        self.attn_norm = RMSNorm(d_model)
        self.attention = GroupedQueryAttention(d_model, n_heads, n_kv_heads)
        self.ffn_norm = RMSNorm(d_model)
        self.ffn = SwiGLU(d_model, ffn_hidden_dim)

    def forward(self, x, rope_cos, rope_sin):
        x = x + self.attention(self.attn_norm(x), rope_cos, rope_sin) # residual
```

```
+ attention
    x = x + self.ffn(self.ffn_norm(x)) # residual +
FFN
    return x
```

The residual pattern (critical for deep networks):



- 1. **Normalize** the input (RMSNorm) — stabilize the distribution before processing
- 2. **Transform** through the sublayer (attention or FFN) — learn a *delta* (change)
- 3. **Add back** to the original input — the residual connection

Why this works: Each block only needs to learn the *difference* from what's already there (*delta learning*). The original signal flows through the skip connection untouched. This prevents the vanishing gradient problem — gradients can backpropagate through the skip path even if the sublayer's gradients are small.

6.2 Full Model: MiniLLM

```
class MiniLLM(nn.Module):
    def __init__(...):
        self.token_emb = nn.Embedding(vocab_size, d_model)
        self.layers = nn.ModuleList([TransformerBlock(...) for _ in
range(n_layers)])
        self.final_norm = RMSNorm(d_model)
        self.lm_head = nn.Linear(d_model, vocab_size, bias=False)
        self.lm_head.weight = self.token_emb.weight # <- weight tying!
```

Key design decisions:

Decision	Explanation
No positional embedding table	RoPE handles position <i>inside</i> attention — no need for a separate table
nn.ModuleList of blocks	Allows variable depth; each block has its own parameters
final_norm before lm_head	Stabilizes the final representation before classification
Weight tying	See below — saves ~16K params and improves quality

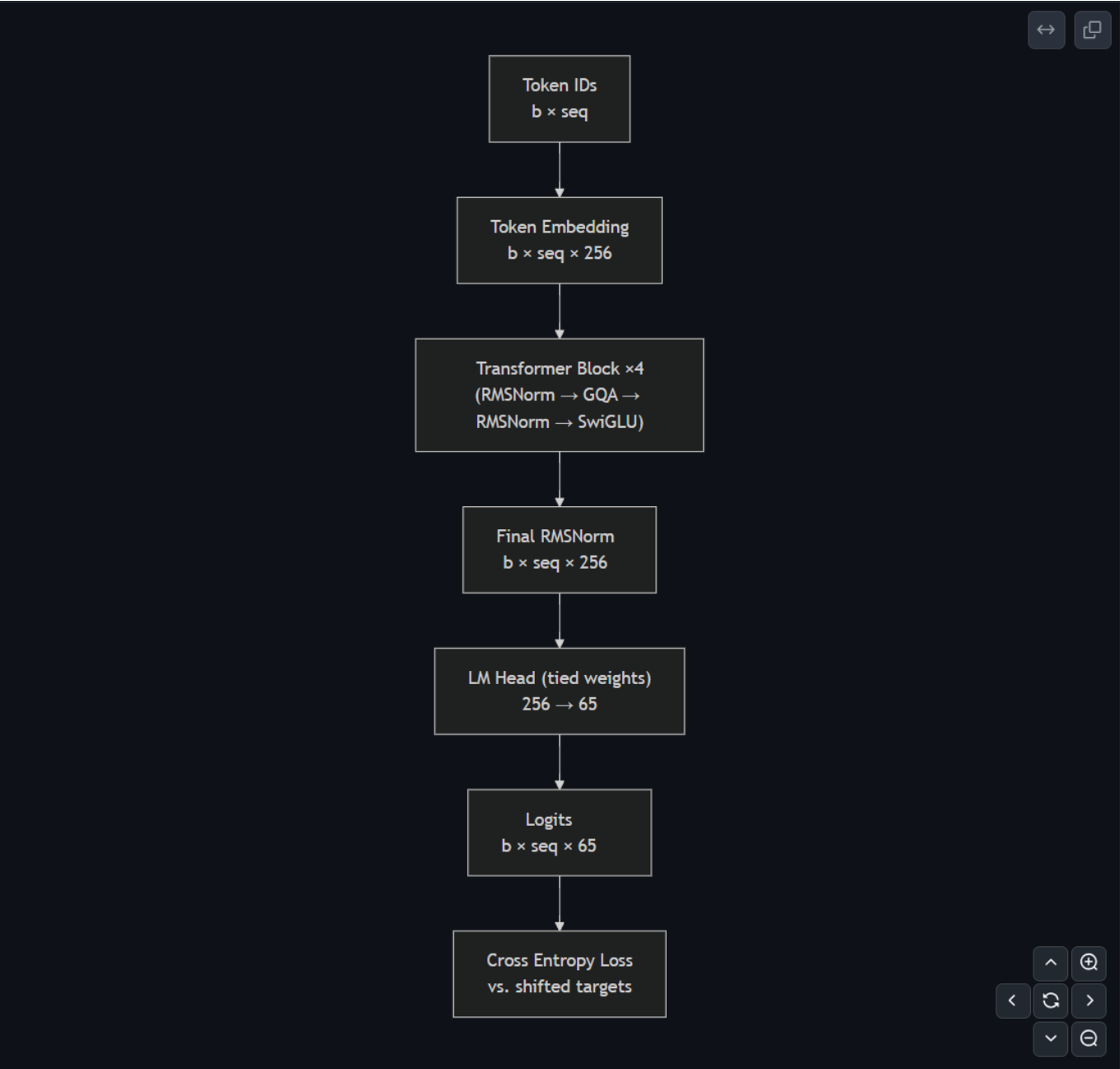
Weight tying explained:

```
self.lm_head.weight = self.token_emb.weight # Same matrix, transposed use
```

- The **embedding layer** maps token IDs → dense vectors (lookup: $d_model \times vocab_size$)
- The **LM head** maps dense vectors → logits over tokens (projection: $vocab_size \times d_model$)
- These are inverse operations — sharing the same weight matrix makes them learn a **consistent** token space
- Saves $65 \times 256 = 16,640$ parameters and acts as a regularizer

Analogy: Weight tying is like using the same dictionary to both translate *from* English to a secret code (embedding) and *back from* the code to English (LM head). Forcing both directions to use the same dictionary means the code stays internally consistent.

6.3 End-to-End Architecture



6.4 Shape Trace for One Forward Pass

Stage	Shape	Notes
-------	-------	-------

Stage	Shape	Notes
Input token IDs	[b, seq]	Integer indices in [0, 64]
Token embedding	[b, seq, 256]	Lookup from embedding table
Q in attention	[b, 8, seq, 32]	8 query heads × 32 dims each
K/V before repeat	[b, 2, seq, 32]	Only 2 KV heads (GQA)
K/V after repeat	[b, 8, seq, 32]	Expanded to match 8 Q heads
Attention scores	[b, 8, seq, seq]	$Q \cdot K^T / \sqrt{32}$, masked
Attention output (merged)	[b, seq, 256]	Heads concatenated + projected
SwiGLU intermediate	[b, seq, 680]	Expanded hidden dim
SwiGLU output	[b, seq, 256]	Projected back down
Final logits	[b, seq, 65]	One score per vocab token
Loss computation	logits [b·seq, 65], targets [b·seq]	Flattened for cross_entropy

6.5 Parameter Breakdown

Observed model summary:

```
Total params:      2,763,264
Trainable params: 2,763,264
Model size (fp32): ~11.1 MB
```

7. Training and Evaluation

7.1 Hyperparameters

```
BATCH_SIZE      = 64
CONTEXT_LEN     = 256
LEARNING_RATE   = 3e-4
MAX_STEPS       = 3000
EVAL_INTERVAL   = 250
EVAL_STEPS      = 20
LOG_INTERVAL    = 50
optimizer = torch.optim.AdamW(model.parameters(), lr=LEARNING_RATE)
```

Per-step compute budget:

- Tokens per step: 64 × 256 = 16,384
- Total tokens seen: 3000 × 16,384 = 49,152,000 (~49× the dataset, or ~49 epochs)
- Why AdamW: adaptive learning rates per parameter + decoupled weight decay

7.2 Training Loop — The 6-Step Rhythm

```
for step in range(MAX_STEPS):
    xb, yb = get_batch("train", BATCH_SIZE, CONTEXT_LEN) # 1. Fetch batch
    logits, loss = model(xb, yb) # 2. Forward pass
    optimizer.zero_grad() # 3. Zero gradients
    loss.backward() # 4. Backward pass
    torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0) # 5. Clip
    gradients
    optimizer.step() # 6. Update weights
```

Step	What it does	Why it matters
1. Fetch batch	Random 64 × 256 windows from training data	Stochastic sampling — each step sees different data
2. Forward	Run input through model, compute loss	Cross-entropy loss = how wrong are the predictions?
3. Zero grad	Reset .grad tensors to 0	PyTorch accumulates gradients by default — must reset
4. Backward	Compute $\partial \text{loss} / \partial \text{param}$ for every parameter	Backpropagation through the entire computation graph
5. Clip grad	Cap gradient norm at 1.0	Prevents exploding gradients (one bad batch can't nuke the model)
6. Update	<code>param -= lr × adjusted_grad</code> (AdamW)	Actually move the weights toward lower loss

Gradient clipping detail: If the total gradient norm exceeds 1.0, all gradients are scaled down proportionally. This is a safety valve — without it, a single unlucky batch could produce enormous gradients and catastrophically update the weights.

7.3 Evaluation Function

```
@torch.no_grad()
def estimate_loss():
    model.eval()
    out = {}
    for split in ["train", "val"]:
        losses = torch.zeros(EVAL_STEPS)
        for k in range(EVAL_STEPS):
            xb, yb = get_batch(split, BATCH_SIZE, CONTEXT_LEN)
            _, loss = model(xb, yb)
            losses[k] = loss.item()
        out[split] = losses.mean()
    model.train()
    return out
```



Decorator/Call	Purpose
<code>@torch.no_grad()</code>	Disables gradient computation → saves memory and speeds up inference
<code>model.eval()</code>	Disables dropout and switches batch norm (if any) to inference mode
Average over 20 batches	Reduces noise — a single batch's loss is noisy; averaging stabilizes
<code>model.train()</code> at the end	Re-enables dropout for continued training

7.4 Baseline Run: Actual Logged Metrics

Initial sanity check:

- Expected loss for random uniform prediction over 65 tokens: $\ln(65) \approx 4.17$
- Observed step 0 loss: **4.2273** ☒ (close to random — model hasn't learned anything yet)

Why this sanity check matters: If step-0 loss were much higher or lower than $\ln(\text{vocab_size})$, it would indicate a bug (e.g., wrong loss function, label mismatch, or numerical issues).

Training progression:

Step	Train Loss	Val Loss	Interpretation
0	4.2273	—	Random initialization ($\approx \ln 65$)
250	1.6231	1.7724	Rapid learning phase; both dropping fast
500	1.3927	1.6053	Still improving; gap widening slightly
1000	1.2622	1.5130	Val loss slowing down
1500	1.1686	1.4770	Val starting to plateau
2000	1.1103	1.4939	 Val loss starting to rise
2500	1.0639	1.4926	Train still improving; val stagnant
2999	1.0163	1.5086	Clear overfitting gap

Runtime: 673 seconds (11.2 min) on Tesla T4

Interpretation of the loss curves:

- Steps 0–1000:** Both losses drop rapidly — the model is learning genuine patterns (character frequencies, common words, dialog structure)
- Steps 1000–1500:** Validation loss plateaus while training loss keeps dropping — the model is starting to memorize training-specific patterns
- Steps 1500–3000:** Classic **overfitting** — train loss continues to decrease (memorization), val loss slightly increases (generalization degraded)

8. Generation (Autoregressive Sampling)

8.1 The Generation Loop

```
@torch.no_grad()
def generate(model, prompt, max_new_tokens=500, temperature=0.8):
    model.eval()
    tokens = encode(prompt)
    tokens = torch.tensor(tokens, dtype=torch.long, device=device).unsqueeze(0)

    for _ in range(max_new_tokens):
        context = tokens[:, -config["max_seq_len"]:] # sliding window
        logits, _ = model(context)
        logits = logits[:, -1, :] / temperature # scale last position
        probs = F.softmax(logits, dim=-1)
        next_token = torch.multinomial(probs, num_samples=1) # sample
        tokens = torch.cat([tokens, next_token], dim=1)
    return decode(tokens[0].tolist())
```

Step-by-step pipeline:

Step	Code	What it does
1	<code>encode(prompt)</code>	Convert prompt string to token IDs
2	<code>tokens[:, -max_seq_len:]</code>	Keep only the last 256 tokens (sliding window)
3	<code>model(context)</code>	Forward pass → logits for every position
4	<code>logits[:, -1, :]</code>	Take only the last position's logits
5	<code>/ temperature</code>	Scale logits — lower T = sharper, higher T = flatter
6	<code>F.softmax(...)</code>	Convert scaled logits to probabilities
7	<code>torch.multinomial(...)</code>	Sample one token from the distribution
8	<code>torch.cat(...)</code>	Append sampled token and repeat

8.2 Temperature — The Creativity Dial

Temperature divides logits *before* softmax, controlling the distribution's "sharpness":

Temperature	Effect	Typical Output
0.1–0.3	Extremely peaked → almost always picks top token	Very repetitive, deterministic
0.5	Peaked but with some variety	Coherent, conservative, repetitive
0.8	Balanced	Varied, still readable (default)
1.0	Raw model probabilities (unchanged)	Noisier, more surprising
1.2+	Flattened — low-prob tokens get boosted	Creative but unstable, gibberish risk

Analogy: Temperature is like a radio dial between "boring but safe" (low) and "wild but risky" (high). At $T=0.5$, the model plays greatest hits. At $T=1.2$, it goes full jazz improvisation.

Math intuition:

- At $T \rightarrow 0$: $\text{softmax}(\text{logits}/T) \rightarrow$ one-hot on the argmax (greedy)
- At $T = 1$: original model distribution
- At $T \rightarrow \infty$: $\rightarrow$ uniform distribution (pure random)

8.3 Example Outputs (from notebook run)

Prompt: ROMEO:

- $T=0.5$: Most structured pseudo-Shakespeare; repetitive but grammatically plausible
- $T=0.8$: Varied, still somewhat readable; good dialog flow
- $T=1.2$: Highly chaotic and noisy; occasional creative fragments

Prompt: RISHABH:

- Model still responds in Shakespearean dialog style — demonstrates **dataset conditioning**
- It has internalized the pattern `CHARACTER_NAME:\nDialog text...` regardless of whether the name exists in the training data

8.4 What's Missing (and Why It Matters)

Feature	What it does	Why it's not here
KV Cache	Cache K/V from previous tokens	Would make generation $\sim \text{seq_len} \times$ faster
Top-k sampling	Only sample from top k tokens	Cuts off garbage tail
Top-p (nucleus)	Sample from tokens covering p% cumulative prob	Adaptive cutoff, better than fixed k

9. Debugging and Sanity Checks

9.1 First-Run Checklist ☒

#	Check	Expected	Red Flag If...
1	Initial loss	$\approx \ln(\text{vocab_size}) = 4.17$	Much higher $\rightarrow$ loss bug; much lower $\rightarrow$ label leak
2	Target shift	$y = x$ shifted by 1	$y[t] \neq x[t+1] \rightarrow$ batching bug
3	Head divisibility	$d_model \% n_heads == 0$	Non-zero remainder $\rightarrow$ shape crash
4	GQA divisibility	$n_heads \% n_kv_heads == 0$	Non-zero $\rightarrow$ <code>repeat_kv</code> fails

#	Check	Expected	Red Flag If...
5	RoPE head dim	Must be even	Odd → dimension mismatch in apply_rope
6	Causal mask	Upper triangle = <code>-inf</code>	No mask → future token leakage

9.2 Common Bugs and Fixes

Symptom	Likely Issue	Fix
Shape errors in attention	Bad head reshape/transpose	Print <code>q/k/v</code> shapes; ensure <code>.view(b, seq, heads, dim).transpose(1,2)</code>
Loss becomes NaN	Unstable gradients / bad LR	Lower LR, keep grad clipping, check for div-by-zero
Good train, poor val	Overfitting	More data, stronger regularization, early stop
Generation repeats chars	Too low temperature or undertrained	Raise temperature, train longer
CUDA OOM	Batch × context too large	Lower batch size or sequence length
Loss plateaus at ~2.0	Model too small	Increase <code>d_model</code> or <code>n_layers</code>

9.3 Quick Debugging Snippets

```
# Check for NaN in parameters
for name, p in model.named_parameters():
    if torch.isnan(p).any(): print(f"NaN in {name}")

# Verify target shift
x, y = get_batch("train", 1, 8)
assert (x[0, 1:] == y[0, :-1]).all(), "Target shift is wrong!"

# Check initial loss
model.eval()
_, loss = model(*get_batch("train", 32, 256))
print(f"Initial loss: {loss.item():.4f}, expected: {math.log(65):.4f}")
```

10. Extension Roadmap

🔑 High Impact, Low Effort

Upgrade	What to do	Expected benefit
LR schedule	Warmup (100 steps) + cosine decay	Better convergence, less overfitting

Upgrade	What to do	Expected benefit
Mixed precision	<code>torch.cuda.amp.autocast()</code>	~2× training speed on T4
KV cache	Store K/V in generation loop	~seq_len× faster generation

🔧 High Impact, Medium Effort

Upgrade	What to do	Expected benefit
BPE tokenizer	SentencePiece / tiktoken	Much higher quality text output
Top-k / top-p sampling	Filter low-prob tokens before sampling	Cleaner generation
Checkpointing	Save model every N steps	Robustness, enables experimentation

🔧 High Impact, High Effort

Upgrade	What to do	Expected benefit
Scale model	Increase <code>d_model</code> , <code>n_layers</code> , context	Better quality (needs more data too)
Better dataset	Domain-cleaned, consistent style corpus	Higher quality ceiling
Multi-GPU	<code>DistributedDataParallel</code>	Scale beyond single GPU

11. Quick Revision Card

11.1 Core Formulas

11.1 Core Formulas

RMSNorm: $\text{RMS}(x) = \sqrt{\frac{1}{d} \sum x_i^2 + \epsilon}$ $y = \frac{x}{\text{RMS}(x)} \odot \gamma$

Scaled Dot-Product Attention: $\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_h}}\right)V$

RoPE (per dimension pair):

$$x'_{2i} = x_{2i}\cos\theta - x_{2i+1}\sin\theta$$
$$x'_{2i+1} = x_{2i}\sin\theta + x_{2i+1}\cos\theta$$

SwiGLU: $\text{SwiGLU}(x) = W_{\text{down}}\left(\text{SiLU}(W_{\text{gate}}x) \odot W_{\text{up}}x\right)$

11.2 Canonical Model Numbers

Item	Value
Vocab	65
Layers	4
d_model	256
Q heads	8
KV heads	2
Head dim	32
FFN hidden	680
Context length	256
Total params	2,763,264

11.3 Canonical Training Numbers

Metric	Value
Steps	3,000
Batch size	64
LR	3e-4
Optimizer	AdamW
Grad clip	1.0
Dropout	0.2
Runtime	673s
Final train loss	1.0163
Final val loss	1.5086

11.4 One-Line Summaries

Component	One-Liner
RMSNorm	Normalize by root-mean-square, skip mean subtraction — simpler, just as good
RoPE	Rotate Q/K dimension pairs by position → dot products encode relative distance
GQA	Share KV heads across query groups → 4× KV cache savings
SwiGLU	Gated FFN with smooth activation → learned feature filtering, no dead neurons
Pre-norm	Normalize → transform → add back → stable gradients through deep stacks
Weight tying	Embedding and LM head share weights → consistent token space, fewer params

Closing Summary

This Week 07 implementation is a compact but **faithful miniature of modern decoder-only LLM internals**:

What we built	What production LLMs add
☒ Pre-norm RMSNorm	Same — it's identical
☒ RoPE	NTK-aware scaling, YaRN for longer contexts
☒ GQA	Flash Attention, paged KV cache
☒ SwiGLU	Same structure, much larger hidden dims
65-token char vocab	32K–128K subword vocab
4 layers, 256 width	32–80 layers, 4096–8192 width
1.1M char corpus	Trillions of tokens

The **architecture is the same. The scale is different.** Once you understand these 2.7M parameters, you understand the skeleton of a 70B-parameter LLaMA.

Made with ❤️ by [Akshat Shethia](#)